AI-Enhanced Programming with Python

Course Syllabus — Fall 2026

Term: Fall 2026 **Format:** AI-Enhanced Learning **Contact:** ssolis1@pacific.edu

Course Description and Philosophy

This semester, we will build Python programming skills and confidence, progressing from foundational coding mechanics to practical applications. This is an interactive, AI-enhanced course. Artificial intelligence may support debugging, help explore alternatives, and personalize practice after foundational skills have been attempted and understood.

Students should treat their work as the development of professional skills and, where appropriate, portfolio artifacts. The goal is not merely to finish an assignment: it is to create, test, explain, document, and improve programs.

Learning and Assessment Activities

Students will complete four kinds of work:

- **In-class activities:** Guided programming activities used to practice and demonstrate weekly concepts.
- **Weekly homework:** Individual practice, often connected to textbook exercises, with written reflection or testing evidence when assigned.
- **Individual book-topic teaching activity:** Each student chooses a topic from the weekly schedule, studies the related textbook material, and builds a Python-based activity in Git that helps classmates understand and practice the concepts.
- **Final application project:** Each student proposes and builds a Python application that uses course skills and can be presented as a professional portfolio artifact.

Grade Weights

Category	Weight
In-class activities	20%
Weekly homework	25%
Individual book-topic presentation	15%
Final application project	40%
Total	**100%**

Each assignment will include or link to its requirements and rubric. Grading focuses on demonstrated learning, correctness, testing, explanation, and completeness rather than on whether a student used AI.

Submission Expectations

Textbook Homework

Submit textbook homework through Canvas as the requested Python source files and written work (for example, a journal, test notes, or reflection). Git is encouraged for version history and learning, but a public Git repository is not required for textbook exercises. Students may use a private practice repository.

In-Class Activities

Maintain in-class programming activities in Git as professional project repositories. Submit the repository link through Canvas using the instructor's directions. Include a README with the project purpose, setup/run instructions, and any special instructions. Include a `requirements.txt` when the program uses third-party Python packages; when it uses only the standard library, state the Python version and run command in the README instead.

Repositories must not contain passwords, API keys, or private personal information. Follow instructor directions for repository visibility and granting access. If a repository link cannot be accessed by the grader, the student may be asked to provide an approved Canvas archive or correct access.

Final Project

Maintain the final application in Git and submit its repository link through Canvas. The project repository must include a professional README, setup/run instructions, appropriate dependency information, testing evidence, and an AI-use statement. Final project instructions will specify milestones, presentation/demo expectations, and the final rubric.

Canvas and Repository Access

Canvas is the official submission channel for assignment links, requested files, written work, and grade feedback. Git is the version-control and project-documentation space for in-class activities and the final application. Unless explicitly approved, do not publish student work publicly. Use the course-approved repository platform and access settings.

Artificial Intelligence Use

AI may be used as a learning and development aid when an assignment permits it. Students remain responsible for their submitted work and should be able to explain and verify it. Follow the specific AI instructions on each assignment.

When asked, disclose meaningful AI assistance by stating the tool used, what help was requested, what suggestion was used or changed, and how the result was checked. Do not submit generated code or explanations you cannot explain. Do not share passwords, API keys, private records, or other sensitive information with AI tools. Verify factual claims and test code rather than assuming AI output is correct.

The course also introduces AI agents: systems that can use tools or take bounded steps toward a goal. Students will consider permissions, privacy, reliability, verification, and human oversight when using or designing agent-based workflows.

Professional Git Practice

For repositories required by an assignment:

- Use clear filenames and an understandable folder structure.
- Commit meaningful increments as work progresses; do not leave all work as one unexplained final commit.
- Write a README for a new reader that explains what the project does and how to run it.
- Include only necessary dependencies and project files.
- Keep credentials and private information out of the repository.
- Credit external code, data, and assets according to their license and the assignment directions.

Final Application Project

Each student will propose and develop a Python application that creates value for a defined user or need. The application should be achievable within the course timeline, demonstrate course skills, handle relevant inputs and edge cases, and include testing and professional documentation. Students will submit the project proposal, checkpoints, repository, README, and final demonstration/reflection as described in the Final Project instructions.

Individual Book-Topic Presentation

Each student chooses a topic represented in the weekly schedule, studies the corresponding textbook material until they can explain it, and creates an engaging Python-based learning activity for classmates. The activity should help peers understand and practice the selected concepts, not just watch a demonstration. Build the activity in a Git repository with a README that lets a classmate set it up, run it, and follow the instructions independently. Test those directions with a peer before presenting. Submit the repository link through Canvas and cite the textbook chapter and printed pages. See the book-topic teaching activity instructions for requirements and rubric.

Weekly Schedule and Milestone Timeline

Weeks 6–7 are intentionally omitted from the topic schedule.

Phase 1: Core Python Foundations

- **Week 1:** Course orientation and technology setup. Introduction to the AI-enhanced framework, IDE setup, and environment/package management. Compare AI assistants and agents; discuss tools, permissions, privacy, verification, and human oversight.
- **Week 2:** Murach's Python Programming, Section 1, Part A. Essential concepts and skills. **Facilitator:** Shyla C. Solis.
- **Week 3:** Murach's Python Programming, Section 1, Part B. Operators, conditional logic, and control loops. **Facilitator:** Shyla C. Solis.

Phase 2: Advancing Python Skills

- **Week 4:** Murach's Python Programming, Section 2, Part A. Functions, lists, tuples, and exceptions. **Facilitator:** Wendy Chen.
- **Week 5:** Murach's Python Programming, Section 2, Part B. Dictionaries, strings, and dates/times. **Facilitator:** Wendy Chen.
- **Weeks 6–7:** No course topics are listed for these weeks.

Phase 3: Applications, Interfaces, and Data Entry

- **Week 8:** Murach's Python Programming, Section 4, Part A. Database programming (SQLite/MySQL connectivity and data tables). **Facilitator:** Xavier Ward.
- **Week 9:** Murach's Python Programming, Section 4, Part B. GUI programming. **Facilitator:** Xavier Ward.

Phase 4: Transitioning to Data Science

- **Week 10:** Murach's Python for Data Science, Section 1, Part A. Pandas, NumPy, and essential DataFrames. **Facilitator:** To be assigned.
- **Week 11:** Murach's Python for Data Science, Section 1, Part B. Data cleaning, filtering rows, and basic plots. **Facilitator:** To be assigned.

Phase 5: Analytical Case Studies

- **Week 12:** Polling Case Study, Part A. Data ingestion and aggregation of polling datasets. **Facilitator:** To be assigned.
- **Week 13:** Polling Case Study, Part B. Date parsing, trend analysis, and communicating results. **Facilitator:** To be assigned.
- **Week 14:** Thanksgiving break; no class. Independent study: refine final project scripts.
- **Week 15:** AI inside the IDE and agent workflows. Use integrated models and explore bounded tool use, human approval, testing, and failure handling in AI-assisted applications. **Facilitator:** Shyla C. Solis.

Assignment due dates, book-topic assignments, and final project milestones will be announced in Canvas.